



ЗВІТ

Лабораторна робота № 7

Дисципліна: Базы даних та інформаційні системи

Тема: Робота з Redis

Виконав

Студент 3 курсу

Групи МІТ-31

Факультету інформаційних технологій

Бабяк Володимир

1. Вступ

Мета роботи: закріпити розуміння роботи Redis та навчитися використовувати його основні можливості.

Redis (Remote Dictionary Server) — це швидка база даних типу "ключ-значення", що зберігає дані в оперативній пам'яті. Вона підтримує різноманітні структури даних: рядки (Strings), списки (Lists), множини (Sets), хеші (Hashes) та відсортовані множини (Sorted Sets).

У даній лабораторній роботі було виконано:

- Встановлення Redis через Docker
- Виконання базових операцій з рядками
- Робота зі структурами даних (List, Set, Hash, Sorted Set)
- Дослідження TTL — часу життя ключів
- Написання міні-програми на Python

2. Виконані завдання

Крок 1 — Встановлення та запуск Redis

Для встановлення Redis використовувався Docker. Було виконано наступні команди:

```
>> docker run --name redis-lab7 -d -p 6379:6379 redis
>>
>> # Перевірити що працює
>> docker ps
>>
>> # Підключитись до redis-cli
>> docker exec -it redis-lab7 redis-cli
>>
>> # Всередині redis-cli:
>> PING
>> # Відповідь: PONG ?
```

```
127.0.0.1:6379> PING
PONG
127.0.0.1:6379>
```

Результат виконання:

Команда	Результат
docker run ...	Контейнер redis-lab7 запущено
docker ps	Контейнер відображається у списку
PING	PONG

Відповідь PONG підтверджує, що Redis сервер працює коректно.

Крок 2 — Операції з рядками

Виконано команди для роботи з рядковими значеннями:

```
127.0.0.1:6379> SET student "Володимир"
OK
127.0.0.1:6379> GET student
"\xd0\x92\xd0\xbe\xd0\xbb\xd0\xbe\xd0\xb4\xd0\xb8\xd0\xbc\xd0\xb8\xd1\x80"
127.0.0.1:6379>
127.0.0.1:6379> SET mycounter 0
OK
127.0.0.1:6379> INCR mycounter
(integer) 1
127.0.0.1:6379> INCR mycounter
(integer) 2
127.0.0.1:6379> GET mycounter
"2"
```

Результати:

Команда	Результат
SET student "Володимир"	OK
GET student	"Володимир"
SET mycounter 0	OK
INCR mycounter (1-й)	(integer) 1
INCR mycounter (2-й)	(integer) 2
GET mycounter	"2"

Команда INCR автоматично збільшує числове значення ключа на 1. Якщо ключ не існує — Redis створює його зі значенням 0 і збільшує до 1.

Крок 3 — Структури даних

Список (List)

```
127.0.0.1:6379> # --- LIST ---
(error) ERR unknown command '#', with args beginning with: '---' 'LIST' '---'
127.0.0.1:6379>
127.0.0.1:6379> LPUSH tasks "Task1"
(integer) 1
127.0.0.1:6379>
127.0.0.1:6379> LPUSH tasks "Task2"
(integer) 2
127.0.0.1:6379>
127.0.0.1:6379> LRange tasks 0 -1
1) "Task2"
2) "Task1"
127.0.0.1:6379>
127.0.0.1:6379> LPOP tasks
"Task2"
127.0.0.1:6379>
```

Команда	Результат
LPUSH tasks "Task1"	(integer) 1
LPUSH tasks "Task2"	(integer) 2
LRange tasks 0 -1	1) "Task2" 2) "Task1"
LPOP tasks	"Task2"

LPUSH додає елементи на початок списку, тому Task2 стоїть першим. LPOP видаляє і повертає перший елемент.

Множина (Set):

```
127.0.0.1:6379> # --- SET ---
(error) ERR unknown command '#', with args beginning with: '---' 'SET' '---'
127.0.0.1:6379>
127.0.0.1:6379> SADD tech:set "Redis"
(integer) 1
127.0.0.1:6379>
127.0.0.1:6379> SADD tech:set "PostgreSQL"
(integer) 1
127.0.0.1:6379>
127.0.0.1:6379> SADD tech:set "MongoDB"
(integer) 1
127.0.0.1:6379>
127.0.0.1:6379> SMEMBERS tech:set
1) "Redis"
2) "PostgreSQL"
3) "MongoDB"
127.0.0.1:6379>
127.0.0.1:6379> SISMEMBER tech:set "Redis"
(integer) 1
```

Команда	Результат
SADD tech:set "Redis"	(integer) 1
SADD tech:set "PostgreSQL"	(integer) 1
SADD tech:set "MongoDB"	(integer) 1
SMEMBERS tech:set	"Redis", "PostgreSQL", "MongoDB"
SISMEMBER tech:set "Redis"	(integer) 1 — елемент існує

Set не зберігає порядок елементів і не допускає дублікатів. SISMEMBER повертає 1 якщо елемент є, 0 якщо немає.

Хеш (Hash):

```
127.0.0.1:6379> # --- HASH ---
(error) ERR unknown command '#', with args beginning with: '---' 'HASH' '---'
127.0.0.1:6379>
127.0.0.1:6379> HSET profile name "Володимир"
(integer) 1
127.0.0.1:6379>
127.0.0.1:6379> HSET profile city "Київ"
(integer) 1
127.0.0.1:6379>
127.0.0.1:6379> HGET profile name
"\xd0\x92\xd0\xbe\xd0\xbb\xd0\xbe\xd0\xb4\xd0\xb8\xd0\xbc\xd0\xb8\xd1\x80"
127.0.0.1:6379>
127.0.0.1:6379> HGETALL profile
1) "name"
2) "\xd0\x92\xd0\xbe\xd0\xbb\xd0\xbe\xd0\xb4\xd0\xb8\xd0\xbc\xd0\xb8\xd1\x80"
3) "city"
4) "\xd0\x9a\xd0\xb8\xd1\x97\xd0\xb2"
```

Команда	Результат
HSET profile name "Володимир"	(integer) 1
HSET profile city "Київ"	(integer) 1
HGET profile name	"Володимир"
HGETALL profile	name "Володимир", city "Київ"

Hash зберігає об'єкти у вигляді пар поле-значення. Зручно для збереження профілів користувачів.

Відсортована множина (Sorted Set):

```

127.0.0.1:6379> # --- SORTED SET ---
(error) ERR unknown command '#', with args beginning with: '---' 'SORTED' 'SET' '---'
127.0.0.1:6379>
127.0.0.1:6379> ZADD scores 85 "Student1"
(integer) 1
127.0.0.1:6379>
127.0.0.1:6379> ZADD scores 92 "Student2"
(integer) 1
127.0.0.1:6379>
127.0.0.1:6379> ZADD scores 74 "Student3"
(integer) 1
127.0.0.1:6379>
127.0.0.1:6379> ZREVRANGE scores 0 -1 WITHSCORES
1) "Student2"
2) "92"
3) "Student1"
4) "85"
5) "Student3"
6) "74"

```

Команда	Результат
ZADD scores 85 "Student1"	(integer) 1
ZADD scores 92 "Student2"	(integer) 1
ZADD scores 74 "Student3"	(integer) 1
ZREVRANGE ... WITHSCORES	Student2(92), Student1(85), Student3(74)
ZRANK scores "Student1"	(integer) 1 — позиція в рейтингу

Sorted Set автоматично сортує елементи за числовим балом (score). ZREVRANGE виводить від найбільшого до найменшого.

Крок 4 — Робота з TTL

Досліджено механізм автоматичного видалення ключів:

```
127.0.0.1:6379> SET temp:data "Hello" EX 10
OK
127.0.0.1:6379> GET temp:data
"Hello"
127.0.0.1:6379> TTL temp:data
(integer) 6
127.0.0.1:6379> GET temp:data
(nil)
127.0.0.1:6379>
```

Команда	Результат
SET temp:data "Hello" EX 10	OK
GET temp:data	"Hello"
TTL temp:data	(integer) 6 — залишок секунд
GET temp:data (після 10с)	(nil) — ключ видалено

Параметр EX встановлює час життя ключа в секундах. TTL повертає скільки секунд залишилось. Значення -2 означає що ключ не існує.

Крок 5 — Міні-програма на Python

Написано скрипт `redis_demo.py`, який виконує всі операції з Redis через бібліотеку `redis-py`:

lab7-redis > redis_demo.py > ...

```
1  import redis
2  import time
3
4  r = redis.Redis(host='localhost', port=6379, decode_responses=True)
5
6  print("=== 1. Перевірка з'єднання ===")
7  print(r.ping()) # True
8
9  print("\n=== 2. Рядки ===")
10 r.set("student", "Володимир")
11 print(f"student = {r.get('student')}")
12 r.set("mycounter", 0)
13 r.incr("mycounter")
14 r.incr("mycounter")
15 print(f"mycounter = {r.get('mycounter')}")
16
17 print("\n=== 3. Список (List) ===")
18 r.delete("tasks")
19 r.lpush("tasks", "Task1", "Task2")
20 print(f"tasks = {r.lrange('tasks', 0, -1)}")
21 print(f"lpop = {r.lpop('tasks')}")
22
23 print("\n=== 4. Множина (Set) ===")
24 r.delete("tech:set")
25 r.sadd("tech:set", "Redis", "PostgreSQL", "MongoDB")
26 print(f"members = {r.smembers('tech:set')}")
27 print(f"is Redis member = {r.sismember('tech:set', 'Redis')}")
28
29 print("\n=== 5. Хеш (Hash) ===")
30 r.hset("profile", mapping={"name": "Володимир", "city": "Київ"})
31 print(f"name = {r.hget('profile', 'name')}")
32 print(f"all = {r.hgetall('profile')}")
33
34 print("\n=== 6. Sorted Set ===")
35 r.zadd("scores", {"Student1": 85, "Student2": 92, "Student3": 74})
36 print(f"ranking = {r.zrevrange('scores', 0, -1, withscores=True)}")
37 print(f"rank of Student1 = {r.zrank('scores', 'Student1')}")
38
39 print("\n=== 7. TTL ===")
40 r.set("temp:data", "Hello", ex=10)
41 print(f"value = {r.get('temp:data')}")
42 print(f"ttl = {r.ttl('temp:data')} сек")
43 time.sleep(11)
44 print(f"нісця 11с: {r.get('temp:data')}") # None
```

```

PS C:\DB Lab 7-9\lab7-redis> python redis_demo.py
• === 1. Перевірка з'єднання ===
True

=== 2. Рядки ===
student = Володимир
mycounter = 2

=== 3. Список (List) ===
tasks = ['Task2', 'Task1']
lpop = Task2

=== 4. Множина (Set) ===
members = {'MongoDB', 'Redis', 'PostgreSQL'}
is Redis member = 1

=== 5. Хеш (Hash) ===
name = Володимир
all = {'name': 'Володимир', 'city': 'Київ'}

=== 6. Sorted Set ===
ranking = [('Student2', 92.0), ('Student1', 85.0), ('Student3', 74.0)]
rank of Student1 = 1

=== 7. TTL ===
value = Hello
ttl = 10 сек
після 11с: None
○ PS C:\DB Lab 7-9\lab7-redis> 

```

3. Аналіз результатів

В ході виконання лабораторної роботи було практично досліджено основні можливості Redis:

- Redis працює як сховище даних типу "ключ-значення" в оперативній пам'яті, що забезпечує надвисоку швидкість читання і запису.
- Різні структури даних (List, Set, Hash, Sorted Set) дозволяють вирішувати різноманітні задачі без додаткової обробки даних.
- Механізм TTL дозволяє автоматично видаляти застарілі дані, що критично для кешування та сесій.
- Python-бібліотека redis-ру забезпечує зручний інтерфейс для роботи з Redis у програмах.

Де можна використати Redis у реальних проєктах:

Сценарій	Опис
Кешування	Зберігання результатів SQL-запитів для прискорення відповіді
Сесії користувачів	Зберігання JWT токенів або даних сесії з TTL
Черги завдань	Celery + Redis для фонових задач
Рейтинги	Sorted Set для таблиць лідерів у іграх
Pub/Sub	Передача повідомлень між мікросервісами в реальному часі
Лічильники	Підрахунок переглядів, лайків через INCR

4. Відповіді на запитання для самоперевірки

1. Як встановити Redis у Windows/Linux?

Windows: через Docker (docker run --name redis -d -p 6379:6379 redis) або WSL.

Linux: sudo apt update && sudo apt install redis-server -y

macOS: brew install redis

2. Які основні команди для роботи з рядками?

Команда	Опис
SET key value	Записати значення
GET key	Отримати значення
INCR key	Збільшити число на 1
DEL key	Видалити ключ
EXISTS key	Перевірити існування

3. Як працювати зі структурами даних?

- List: LPUSH/RPUSH для додавання, LRange для перегляду, LPOP/RPOP для видалення

- Set: SADD для додавання, SMEMBERS для перегляду, SISMEMBER для перевірки
- Hash: HSET для запису, HGET/HGETALL для читання
- Sorted Set: ZADD для додавання з балом, ZREVRANGE для рейтингу, ZRANK для позиції

4. Як встановити час життя ключа?

Під час створення: SET key value EX 10 (10 секунд)

Після створення: EXPIRE key 10

Перевірка залишку: TTL key (повертає секунди, -1 якщо немає TTL, -2 якщо ключ відсутній)

5. Сценарії використання Redis у реальних застосунках

- Кешування запитів до бази даних — зменшення навантаження на PostgreSQL/MySQL
- Зберігання сесій — швидкий доступ до даних авторизованих користувачів
- Черги завдань — асинхронна обробка через Celery або Bull.js
- Pub/Sub — обмін подіями між мікросервісами
- Геопросторові запити — зберігання координат через GEOADD

5. Висновок

У ході виконання лабораторної роботи було успішно встановлено та налаштовано Redis через Docker, вивчено основні команди для роботи з різними типами даних, досліджено механізм TTL для автоматичного видалення ключів.

Написано Python-програму, яка демонструє практичне використання Redis: з'єднання, операції з рядками, списками, множинами, хешами, відсортованими множинами та TTL.

Redis є потужним інструментом для прискорення веб-застосунків завдяки зберіганню даних у пам'яті та підтримці різноманітних структур даних.